Online Retail Sales Analysis

A data analysis project using Python, MySQL and Power BI

Project report

Prepared by: Krishna Chandel
Location: Lucknow, Uttar Pradesh

Tools used

Python (Pandas), MySQL Workbench and Power BI Desktop

This report describes the work I carried out on the Online Retail dataset, from preparing the data to exploring it with SQL and presenting the results in Power BI.

Portfolio project

1. Introduction

I chose the Online Retail dataset for this project because it contains real transaction-style records and offers several useful questions to explore. I wanted to practice the complete process of working with data—not just create charts, but first understand the data, clean it, analyze it, and then present the results.

The project was completed in three main stages. I used Python and Pandas to prepare the dataset, MySQL to answer business questions, and Power BI to build a visual report. Each tool had a different role in the project.

What I wanted to understand

- Which products contribute the most to sales?
- How do sales change from month to month?
- Which customers contribute the most revenue, and who places repeat orders?
- How does sales performance differ between countries?
- Can the data be prepared consistently enough to support reporting?

2. Dataset

The dataset is the UCI Online Retail dataset, which is also commonly shared through Kaggle. It contains transaction records from a UK-based online retailer that sells gift items. The data includes invoice numbers, product codes and descriptions, quantities, prices, invoice dates, customer IDs, and countries.

Some of the main columns used in the analysis were:

- InvoiceNo — identifies an invoice and helps detect cancellations.
- StockCode and Description — identify the products.
- Quantity and UnitPrice — used to calculate line-level sales.
- InvoiceDate — used for monthly and weekday analysis.
- CustomerID — used for customer-level analysis.
- Country — used to compare markets.

The raw data needed some preparation before it could be used reliably. For example, it contained missing values, duplicate records, cancellation indicators, and negative quantities.

3. Cleaning the data with Python

I started by loading the CSV file into Pandas using ISO-8859-1 encoding. I checked the number of rows and columns, data types, unique values, date range, and the number of distinct invoices, products, customers, and countries. This gave me an initial understanding of the dataset before making changes.

Data quality checks

- Checked missing values in each column.
- Counted exact duplicate rows.
- Looked for negative quantities and zero or negative unit prices.
- Checked invoice numbers beginning with “C”, which are used as cancellation indicators in this dataset.
- Reviewed which countries contributed the most records.

Cleaning steps

- Converted InvoiceDate to datetime format.
- Removed exact duplicate rows.
- Trimmed extra spaces from product descriptions.
- Checked missing CustomerID and Description values.
- Created a separate transaction status label for Sale, Return, and Cancellation.
- Removed completely empty rows and standardized whitespace-only cells before exporting the cleaned file.

New columns

I created a revenue column by multiplying Quantity by UnitPrice. I also extracted the year, month, month name, and weekday from InvoiceDate. These columns made it easier to group the transactions and look for trends over time.

Revenue = Quantity × UnitPrice

The final cleaned file was exported as OnlineRetail_Cleaned_Final.csv. The original raw file was kept unchanged.

A note on missing values

In the export step, remaining missing values were filled with the text “NULL”. This is how the submitted cleaning script handles missing cells. When importing the file into a database, it is important to check the data types because a text placeholder can affect numeric or date columns.

4. Exploring the data with Pandas

After cleaning, I used groupby(), sorting, filtering, and aggregation to explore the dataset. The analysis covered sales, products, customers, orders, and countries.

Sales and product analysis

- Calculated total line-level revenue.
- Found products with the highest revenue and products with the highest quantity sold.
- Grouped sales by month and identified the month with the highest sales.
- Calculated each country's contribution to revenue.
- Reviewed transaction status counts and average unit price.

Customer and order analysis

- Counted customers with an available CustomerID.
- Ranked customers by total spending.
- Calculated the average order value by first summing each invoice and then averaging invoice totals.
- Compared the number of unique customers across countries.
- Calculated the combined contribution of the top 10 customers.

Additional analysis

- Month-over-Month revenue growth using pct_change().
- Cumulative revenue over time using cumsum().
- Average invoice value by country.
- Revenue contribution of the top 10 products.
- A country-by-month pivot table to compare performance across markets.

These analyses helped me move beyond basic totals and practice the kind of grouping and comparison that is commonly used in business reporting.

5. SQL analysis in MySQL

Once the data was available in MySQL Workbench, I wrote SQL queries to answer business questions. My SQL file contains 20 questions covering data checks, sales performance, customers, countries, and time-based analysis.

Some of the questions I explored

- How many records, invoices, products, and customers are in the dataset?
- How many cancellations, negative quantities, and non-positive prices are present?
- Which products generate the most revenue?
- Which customers spend the most and which customers place repeat orders?
- How do revenue and order counts compare across countries?
- Which countries have the highest average order value?
- How does monthly revenue compare with the previous month?
- What is the cumulative revenue over time?

SQL concepts used

- GROUP BY and aggregate functions such as SUM(), COUNT(), and AVG().
- CTEs to organize multi-step calculations.
- CASE statements for customer spending categories.
- LAG() to compare a month with the previous month.
- RANK() to rank products within each country.
- SUM() OVER() for cumulative revenue.

For customer segmentation, I used project-defined spending bands: High Spender at 5,000 or above, Medium Spender from 1,000 to below 5,000, and Low Spender below 1,000. These thresholds were chosen for this analysis and are not official customer categories.

6. Power BI dashboard

I used Power BI Desktop to present the analysis in a dashboard. The PBIX file includes a Sales Performance Dashboard and a page titled “Detailed analysis.” The report contains KPI cards and visuals for looking at performance from different angles.

The dashboard includes

- KPI cards for headline measures such as revenue and average order value.
- A line chart to view trends over time.
- A clustered bar chart for comparisons.
- A treemap and donut chart to show contribution or share.
- A matrix-style view and detail table for further investigation.
- A slicer to filter the report.
- A detailed analysis page and a back-navigation button.

How I would use the dashboard

I would begin with the KPI cards to get a quick overview, then check the trend chart to see how performance changes over time. The product and country visuals help compare contribution, while the detailed page can be used to investigate individual records or customer patterns.

The dashboard is included as a PBIX file. Its exact KPI values and screenshots should be added to this report from Power BI before the report is used as a final portfolio submission.

7. What the analysis can help a business understand

The project examines several areas that are useful for retail decision-making:

- Product contribution — identify products that account for a large share of sales.
- Customer contribution — understand how much revenue comes from high-spending customers.
- Repeat purchasing — identify customers who place more than one order.
- Country performance — compare markets by revenue, order count, and average order value.
- Sales trends — observe monthly changes and possible seasonal patterns.
- Returns and cancellations — keep these transactions in mind when interpreting sales totals.

Possible business actions

- Review high-revenue products when planning stock and promotions.
- Consider customer retention campaigns for repeat and high-spending customers.
- Compare revenue with order count and average order value before deciding which countries to prioritize.
- Investigate unusual month-to-month changes before making inventory or marketing decisions.
- Report returns and cancellations separately so they do not get confused with normal sales.

These are possible actions suggested by the analysis areas. They should be checked against the actual results before being presented as firm business recommendations.

8. Limitations and things to check

While working on the project, I identified a few points that need attention when interpreting the results:

- Some records do not have a CustomerID, so customer-level analysis cannot include every transaction.
- Negative quantities may represent returns, while invoice numbers beginning with “C” indicate cancellations. These should be handled consistently.
- Quantity multiplied by UnitPrice gives line-level revenue. It should not automatically be treated as net revenue if returns or cancellations are included.
- Missing values exported as the text “NULL” may need to be converted back to proper nulls during database import.
- Grouping by month name alone can mix different years. Year and month together are better for time-series reporting.
- Revenue should be checked across Python, SQL, and Power BI using the same filters.

Before publishing, I would verify the final totals, compare the revenue calculation across all three tools, and include screenshots of the dashboard pages. This would make the report easier for someone else to review and reproduce.

9. Conclusion

This project gave me practice with the full data analysis workflow. I used Python and Pandas to inspect and clean the data, MySQL to answer business questions, and Power BI to present the analysis visually. Working across these tools helped me understand how data preparation, querying, and reporting fit together in a practical project.

The next improvement would be to add the actual numerical findings and dashboard screenshots, then check that the same revenue and order definitions are used throughout the project. This would make the final report more useful to a recruiter or business reviewer.

Project files

- Python cleaning and analysis documentation.
- MySQL script containing 20 business queries.
- Power BI dashboard (.pbix).
- Cleaned dataset export (OnlineRetail_Cleaned_Final.csv).

Dataset source: UCI Online Retail dataset, commonly distributed through Kaggle. Add the exact Kaggle dataset link to the GitHub README for reproducibility.

Prepared by Krishna Chandel | Online Retail Sales Analysis